inclusionAI / **Ling-3.0-tiny** like 206 Follow inclusionAI 2.6k

Safetensors | bailing_hybrid | custom_code | License: mit

Copy to bucket NEW

Model card | Files xet | Community 6

🤗 Hugging Face | 🤖 ModelScope | 🐙 OpenRouter

Introduction

We are introducing **Ling-3.0-tiny**, a lightweight hybrid reasoning MoE model with **7.9B** total parameters and only **1.3B** activated parameters per token. It is designed to deliver strong reasoning and agentic capabilities at low inference cost, making advanced model capabilities more accessible for local and resource-constrained deployment. BF16, FP8, and INT4 weights are provided for a wide range of hardware and deployment settings.

Key highlights of the model are summarized below:

- **Efficient Hybrid-Linear Architecture:** Ling-3.0-tiny integrates a 3:1 alternating stacking of KDA and MLA (3 Kimi Delta Attention layers

Downloads last month
1,292

Safetensors

Model size | 8B params
Tensor type | F32 · BF16
Chat template | Files info

Inference Providers NEW

This model isn't deployed by any Inference Provider.

Ask for provider support

Model tree for inclusionAI/Ling-...

Quantizations 6 models

Spaces using inclusionAI/Ling... 2

bep40/Space-Mutiverse-Control
AlanGiglio/gaia-final-agent

Collection including inclusionAI/...

Ling 3.0 Collection

followed by 1 Multi-Head Latent Attention layer per 4-layer block) with a sparse MoE FFN comprising 128 routed experts. Each token activates only 8 routed experts and 1 shared expert, allowing the model to balance long-context modeling capability, parameter efficiency, and computational cost.

- **Native Hybrid Reasoning and Agentic Capabilities:** Ling-3.0-tiny supports both fast responses and multi-step reasoning, with thinking mode configurable per request through `enable_thinking`. It delivers balanced performance across general agent tasks, coding, mathematical and scientific reasoning, and instruction following.
- **Local and Edge Deployment:** Designed for efficient local deployment, Ling-3.0-tiny has been validated on **NVIDIA DGX Spark, Apple Silicon MacBook, and Mac mini**, enabling capable reasoning and agentic workloads without datacenter-class GPUs. With FP8, Ling-3.0-tiny reaches around **100-105 tokens/s on DGX Spark** and **86-90 tokens/s on an M4 Pro MacBook**, with approximately **8.34 GiB peak memory usage** at an 8K context length.

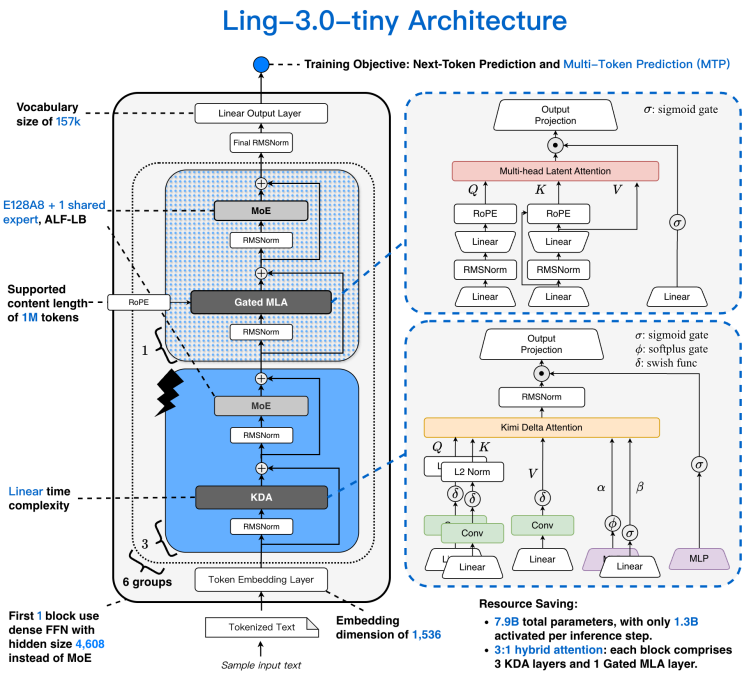
Model Overview

Ling-3.0-tiny inherits the hybrid linear attation architecture of Ling-3.0 series, while being specifically optimized for lightweight and accessible deployment. The model has 7.9B total parameters, with only 1.3B parameters activated per token.

The architecture of Ling-3.0-tiny is designed to make computational efficiency serve real-world agentic performance.

- A 3:1 KDA–MLA architecture (3 KDA layers and 1 MLA layer per 4-layer block) provides more efficient long-context processing;
- A sparse MoE FFN with 128 experts activates 8 routed experts and 1 shared expert per token, enabling broad model capabilities with only 1.3B activated parameters per token.
- Native hybrid reasoning enables fast responses for routine tasks and multi-step reasoning for complex tasks within a single model.

Overall, these designs deliver the inference efficiency needed to deploy lightweight models in real-world agentic workflows.

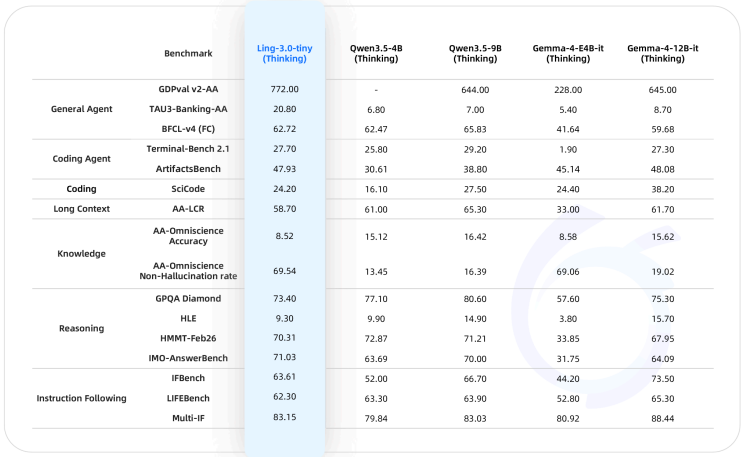


Evaluation

We evaluated Ling-3.0-tiny across agentic tasks, coding, long-context understanding, knowledge

reliability, mathematical and scientific reasoning, and instruction following. Ling-3.0-tiny achieves a score of **25** on the Artificial Analysis Intelligence Index v4.1.1 and **16** on the Artificial Analysis Agentic Index. In Artificial Analysis testing, Ling-3.0-tiny reaches an output speed of over **160 tokens/s**, with approximately **18 seconds** of end-to-end latency for a 500-token response, including reasoning time. These results highlight the model's efficiency relative to its 1.3B activated parameter footprint.

The following table presents representative benchmarks for Ling-3.0-tiny:



	Benchmark	Ling-3.0-tiny (Thinking)	Qwen3.5-4B (Thinking)	Qwen3.5-9B (Thinking)	Gemma-4-E4B-it (Thinking)	Gemma-4-12B-it (Thinking)
General Agent	GDPval v2-AA	772.00	-	644.00	228.00	645.00
	TAU3-Banking-AA	20.80	6.80	7.00	5.40	8.70
	BFCL-v4 (FC)	62.72	62.47	65.83	41.64	59.68
Coding Agent	Terminal-Bench 2.1	27.70	25.80	29.20	1.90	27.30
	ArtifactsBench	47.93	30.61	38.80	45.14	48.08
Coding	SciCode	24.20	16.10	27.50	24.40	38.20
Long Context	AA-LCR	58.70	61.00	65.30	33.00	61.70
Knowledge	AA-Omniscience Accuracy	8.52	15.12	16.42	8.58	15.62
	AA-Omniscience Non-Hallucination rate	69.54	13.45	16.39	69.06	19.02
Reasoning	GPQA Diamond	73.40	77.10	80.60	57.60	75.30
	HLE	9.30	9.90	14.90	3.80	15.70
	HMMT-Feb26	70.31	72.87	71.21	33.85	67.95
	IMO-AnswerBench	71.03	63.69	70.00	31.75	64.09
Instruction Following	IFBench	63.61	52.00	66.70	44.20	73.50
	LIFEBench	62.30	63.30	63.90	52.80	65.30
	Multi-IF	83.15	79.84	83.03	80.92	88.44

> - *Thinking mode is enabled by default. The recommended sampling parameters for Ling-3.0-tiny are `temperature=1.0`, `top_p=0.95`, and `top_k=20`.*
> - *Terminal-Bench 2.1: Evaluated under the Artificial Analysis (AA) protocol using the default Terminus 2 harness, a unified 2-hour timeout, the provided JSON parser in preserve-thinking mode, and 3 runs per task (mean). Decoding uses temperature=1.0, max_new_tokens=32K, with a 256K context window.*

Quickstart

SGLang

The hardware- and recipe-specific launch matrix (BF16/FP8 × Low-Latency / High-Throughput / HiCache + Mooncake), with a live command generator and verified configurations, lives in the SGLang cookbook:

Cookbook: https://docs.sglang.io/cookbook/autoregressive/InclusionAI/Ling-3.0-tiny

Install SGLang

Use the pre-built image that tracks the Ling-3.0 runtime:

```
docker pull lmsysorg/sglang:dev-Ling-3.0-tin
```

Run Inference

Recommended low-latency recipe (built-in MTP / NEXTN, 256K YaRN context) on 1× 141GB-class GPU (H20-3e) or a 1-GPU Blackwell node:

Server

```
docker run --rm --gpus all --ipc=host --shm-
  -p 30000:30000 \
  -e HF_TOKEN=<your-hf-token> \
  lmsysorg/sglang:dev-Ling-3.0-tiny \
  env SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_
  python3 -m sglang.launch_server \
    --model-path inclusionAI/Ling-3.0-tiny \
    --tp 1 \
    --json-model-override-args '{"rope_scali
    --context-length 262144 \
    --speculative-algorithm NEXTN \
```

```
    --mem-fraction-static 0.8 \
    --host 0.0.0.0 \
    --port 30000
```

Client

Thinking is enabled by default by both the chat template and the `ling3` reasoning parser. Disable it per request with `"chat_template_kwargs": {"enable_thinking": false}`. We recommend the sampling parameters `temperature=1.0`, `top_p=0.95`, and `top_k=20`.

```
curl -s http://localhost:30000/v1/chat/compl
  -H "Content-Type: application/json" \
  -d '{"model": "auto",
       "messages": [{"role": "user", "conten
       "stream": true,
       "temperature": 1.0,
       "top_k": 20,
       "top_p": 0.95
     }'
```

For `--reasoning-parser ling3` / `--tool-call-parser ling3`, the HiCache + Mooncake L3 setup, and GSM8K / bench_serving reproduction commands, see the cookbook page linked above.

vLLM

Install vLLM with Ling-3.0 Support

```
pip install uv

uv venv ~/my_ling_env

source ~/my_ling_env/bin/activate
```

```
git clone -b ling_3_0 https://github.com/inc
```

```
cd vllm-ling-v3
```

```
VLLM_USE_PRECOMPILED=1 uv pip install --edit
```

Run Inference

Here is the example to run Ling-3.0-tiny with a single GPU, where the server port is `${PORT}`:

Server

```
vllm serve "$MODEL_PATH" \
    --port "$PORT" \
    --trust-remote-code \
    --served-model-name auto \
    --tensor-parallel-size 1 \
    --gpu-memory-utilization 0.85 \
    --enable-prefix-caching \
    --mamba-cache-mode align \
    --enable-auto-tool-choice \
    --tool-call-parser ling3 \
    --reasoning-parser ling3
```

Client

For better performance, We recommend setting `enable_thinking=true` with `temperature=1.0`, `top_p=0.95`, and `top_k=20`.

```
curl -s http://${MASTER_IP}:${PORT}/v1/chat/
  -H "Content-Type: application/json" \
  -d '{"model": "auto",
       "messages": [{"role": "user", "conten
       "chat_template_kwargs": {"enable_thin
       "stream": true,
       "temperature": 1.0,
```

```
    "top_k": 20,
    "top_p": 0.95
  }'
```

Ollama

> - *This configuration has been verified on an M4 Pro Mac with 48 GB of unified memory.*

Preparation and Build

```
git clone https://github.com/ollama/ollama.g
cd ollama
git fetch origin refs/pull/17643/head:bailin
git switch bailing-moe-v3

cmake -B build .
cmake --build build --parallel 8
```

> - *Support is currently provided by ollama/ollama#17643 and is limited to running via MLX on Apple Silicon.*
> - *Use the local `./ollama` executable built from source in this section. This functionality is not yet included in the official Ollama release.*

Import Model

Replace `/absolute/path/to/bf16_weights` with the absolute path to the BF16 model weights directory. The imported model will be named `ling-tiny-bf16`:

```
printf 'FROM /absolute/path/to/bf16_weights\r
./ollama create ling-tiny-bf16 --experimenta
```

Start Service

Set the default context length to 8192, and then start the Ollama service:

```
# The service listens on http://127.0.0.1:11
OLLAMA_CONTEXT_LENGTH=8192 ./ollama serve
```

Call API

```
curl -sS http://127.0.0.1:11434/api/generate
  "model": "ling-tiny-bf16",
  "prompt": "<role>SYSTEM</role>detailed thi
  "raw": true,
  "think": true,
  "stream": false,
  "options": {
    "temperature": 1.0,
    "top_p": 0.95,
    "top_k": 20,
    "num_predict": 2048
  }
}' | jq -r .response
```